

컴퓨터비전개론_과제#1

1. 파일에서 읽은 영상에 대해 가로 세로를 1/3로 줄인 영상을 생성하는 함수를 작성함. China 영상을 이용하여 결과를 확인

```
import cv2 as cv
import matplotlib.pyplot as plt

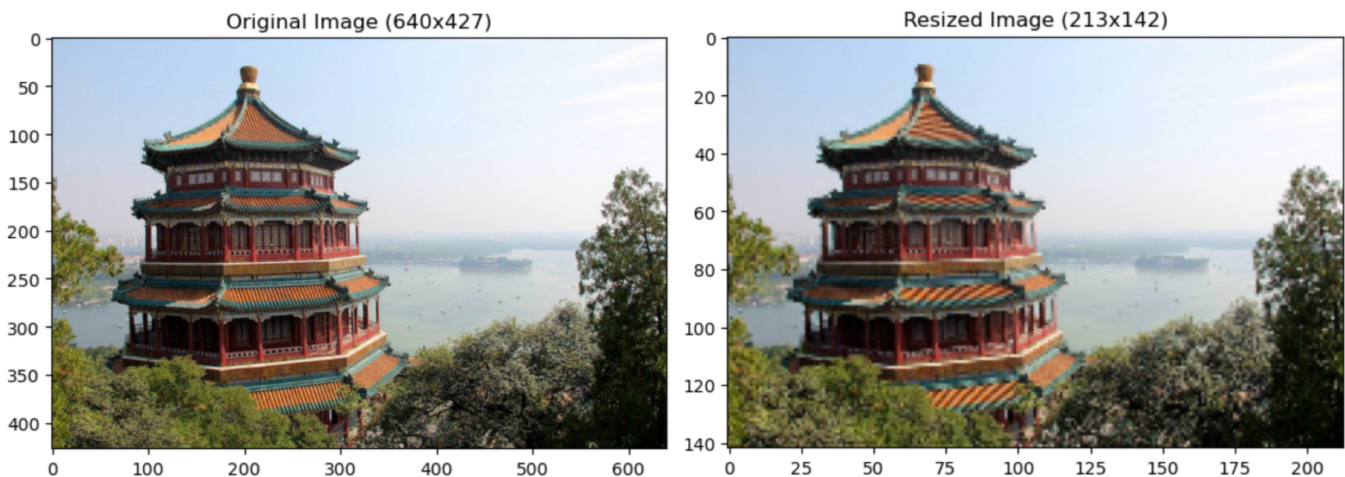
img = cv.imread("china.jpg")

h1, w1 = img.shape[:2] # original image의 크기 정보 저장
plt.imshow(cv.cvtColor(img, cv.COLOR_BGR2RGB))
plt.title(f"Original Image ({w1}x{h1})") # 이미지 출력의 타이틀에 크기 정보를 포함
plt.show()

def resize(img):
    h, w = img.shape[:2] # 높이, 너비까지만 저장 (shape = 높이, 너비, 채널수)
    new_img = cv.resize(img, (int(w / 3), int(h / 3))) # h, w의 입력 순서 바뀜
    return new_img

resized_img = resize(img)

h2, w2 = resized_img.shape[:2] # resized image의 크기 정보 저장
plt.imshow(cv.cvtColor(resized_img, cv.COLOR_BGR2RGB))
plt.title(f"Resized Image ({w2}x{h2})") # 이미지 출력의 타이틀에 크기 정보를 포함
plt.show()
```



우선, OpenCV는 이미지를 BGR 방식으로 불러오고 저장하기 때문에, 이미지를 불러올때 cvtColor() 함수를 사용하여 RGB로 바꿔줬습니다.

원본 이미지인 img의 출력을 보면 640x427 사이즈이고, resize 함수를 적용하여 만든 resized_img의 출력을 보면 213x142로, 원본 이미지에 비해 가로 세로가 1/3로 줄어든 것을 볼 수 있습니다.

resize의 함수 내에서 이미지의 높이 h, 너비 w를 지정 받는데, cv.resize(src,dsize) 에서 dsize는 (width, height) 순서로 입력해야하므로, h와 w의 순서를 바꾼 w/3, h/3 순서로 입력해주어야 합니다.

2. Bilinear(양선형) interpolation

color_rect 함수를 수정하여 오른쪽 그림과 같이 256x256 크기의 color 영상을 **bilinear interpolation**에 의해 생성하는 프로그램을 작성

```
import numpy as np
import matplotlib.pyplot as plt

def color_rect_bilin_interpol():
    height, width = 256, 256
    result = np.zeros((height, width, 3), dtype=np.uint8)

    # 네 모서리 색상 지정 (RGB 기준)
    top_left = np.array([255, 0, 0]) # Red
    top_right = np.array([0, 0, 255]) # Blue
    bottom_left = np.array([0, 255, 0]) # Green
    bottom_right = np.array([255, 255, 0]) # Yellow

    # 각 픽셀 위치마다 색상 계산 (Bilinear Interpolation: 이중선형 보간법)
    for y in range(height):
        fy = y / (height - 1) # y 방향에서의 상대위치 (0~1)
        for x in range(width):
            fx = x / (width - 1) # x 방향에서의 상대위치 (0~1)

            # 1단계: 수평(x축) 방향으로 보간
            top = (1 - fx) * top_left + fx * top_right
            bottom = (1 - fx) * bottom_left + fx * bottom_right

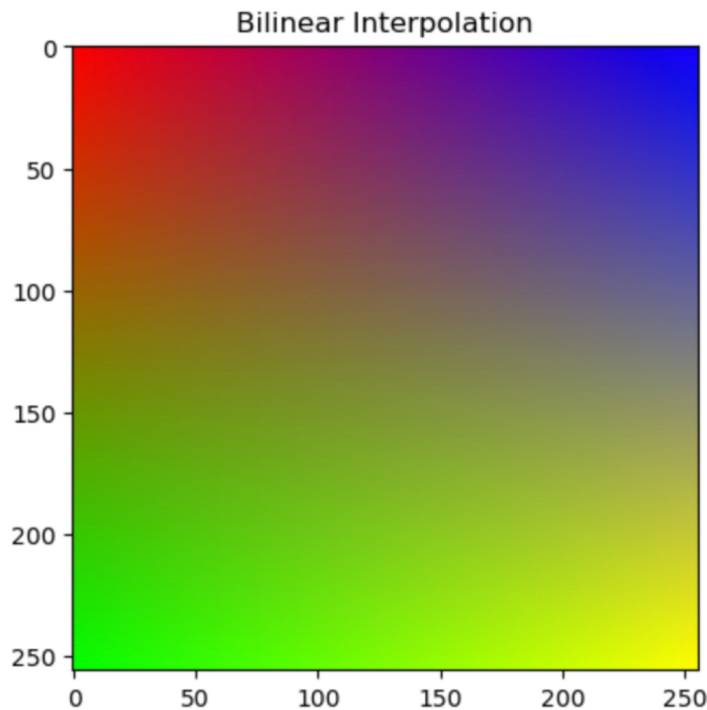
            # 2단계: 수직(y축) 방향으로 보간
            pixel = (1 - fy) * top + fy * bottom

            # 계산된 값을 정수로 변환해서 이미지에 저장
            result[y, x] = pixel.astype(np.uint8)

    return result

img = color_rect_bilin_interpol()

plt.imshow(img)
plt.title("Bilinear Interpolation")
plt.show()
```



1단계

$$top = (1 - f_x) \times top_{left} + f_x \times top_{right}$$

$$bottom = (1 - f_x) \times bottom_{left} + f_x \times bottom_{right}$$

2단계

$$pixel = (1 - f_y) \times top + f_y \times bottom$$

color_rect_bilin_interpol() 함수는 위와 같은 수식에 의해 Bilinear Interpolation(이중선형 보간법)을 동작 시킵니다. 2중 for문을 통해 픽셀의 x축을 옮겨가며 보간을 하고, 다음 y축으로 옮긴 후에 x축을 옮겨가며 보간을 반복하는 구조입니다.

$fx = x / (width - 1)$ 은, x 방향(가로)에서 얼마나 오른쪽으로 갔는지 0~1 사이의 범위를 나타내고,

$fy = y / (height - 1)$ 은, y 방향(세로)에서 얼마나 아래쪽으로 갔는지 0~1 사이의 범위를 나타냅니다.

top은 이미지의 맨 윗줄에 있는 두 점인 top_left, top_right 두 점 사이에서 x 방향으로 fx 만큼 이동한 지점의 색상이고, bottom은 이미지의 맨 아랫줄에 있는 두 점 bottom_left, bottom_right 두 점 사이에서 x 방향으로 fx 만큼 이동한 지점의 색상입니다.

이렇게 계산한 top, bottom 두 값을 y 방향으로 fy 만큼 보간한 결과를 pixel에 저장합니다.

즉, top과 bottom은 y축 기준 윗줄과 아랫줄의 가로 선형 보간 결과이고, 이 두 값을 다시 y축 기준으로 보간해서 최종위치의 색상(=pixel)을 구합니다.